Data Science

Ensemble Models: Bagging & Boosting



DSB Advanced Modeling

GENERAL ASSEMBLY

Lesson roadmap

Welcome + Introduction

Ensemble Methods

Bagging + Out-of-bag Error

Random Forests

Boosting

Bring It Home

Learning objectives



1. Explain why ensembles of trees outperform a single tree (variance and bias).
2. Describe bootstrapping and bagging, and use out-of-bag error.
3. Explain how random forests decorrelate trees, and interpret feature importance.
4. Explain how boosting (AdaBoost, gradient boosting, XGBoost) reduces bias.
5. Fit, compare and choose between ensemble models in scikit-learn.



Warm Up

Same Data, Different Trees

© 2021 General Assembly

DISCUSSIONS: Avoiding overfitting

Avoiding overfitting

- You're training a single decision tree and it overfits badly.
- What are some intuitive ways you might reduce overfitting **without** touching the model architecture?



 © 2025 General Assembly

Ensemble Methods

A problem...

The (white box) models we've seen so far may not be good enough for what we are trying to accomplish.

So what can we do?

We could use **neural networks**, but they require **a lot** of labeled training data to be effective.

One thing we can do to improve performance is to use **ensemble learning**.

 © 2025 General Assembly

Ensemble models

Instead of trying to get one very accurate model, **we can combine the predictions of many weaker models** to make one **ensemble model**.

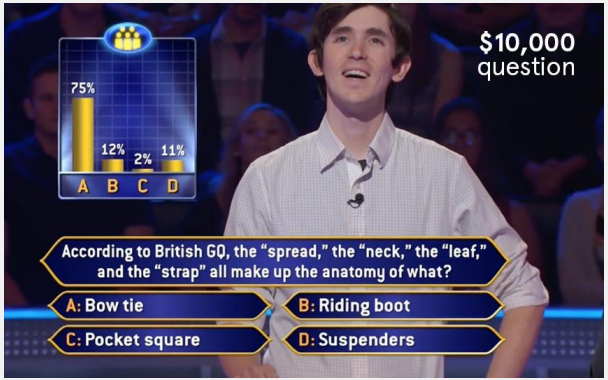
 © 2025 General Assembly

How much does Penelope weigh?

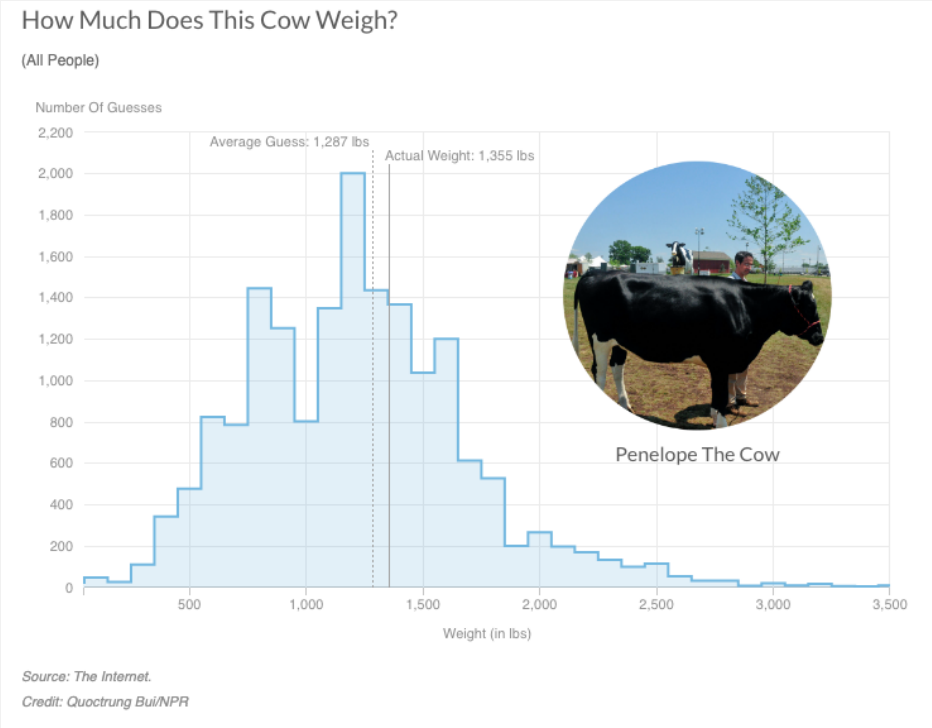


 © 2025 General Assembly

Polling a crowd







 © 2025 General Assembly

The Wisdom of the Crowds

Individuals are dumb, but crowds are smart!

It's an actual statistical fact that, instead of relying on one "ok" model's opinion, you'll get better results if you aggregate many "ok" models together!

This act of **ensembling** **reduces the overall variance** of your model. Since decision trees tend to **overfit**, they're a prime opportunity for ensembling.

While less effective, aggregating "dumb" learners (*i.e.*, *underfit models*) tends not to improve model fit. Reducing bias is boosting's job *(later today)*

 © 2025 General Assembly

How can we create ensemble models?

There are **3 main ensemble learning methods**:

1. Bagging
2. Boosting
3. Stacking

 © 2025 General Assembly

Bagging

Ensemble Methods

Bagging

We can simulate the wisdom of the crowds to improve our “simple” decision trees. The method of **b**ootstrap **agg**regat**ing**, or **bagging**, works as follows:

1. Take a sample of your data
2. Fit a model ***M*** (in our case a decision tree) to that data
3. Repeat this process ***B*** times (often $B = 500$, but it’s arbitrary)

You now have a set of models $M_1, ..., M_B$.

How can we use it?

 © 2025 General Assembly

Bagging Prediction

For a new sample, compute its predictions **for each model you have**:

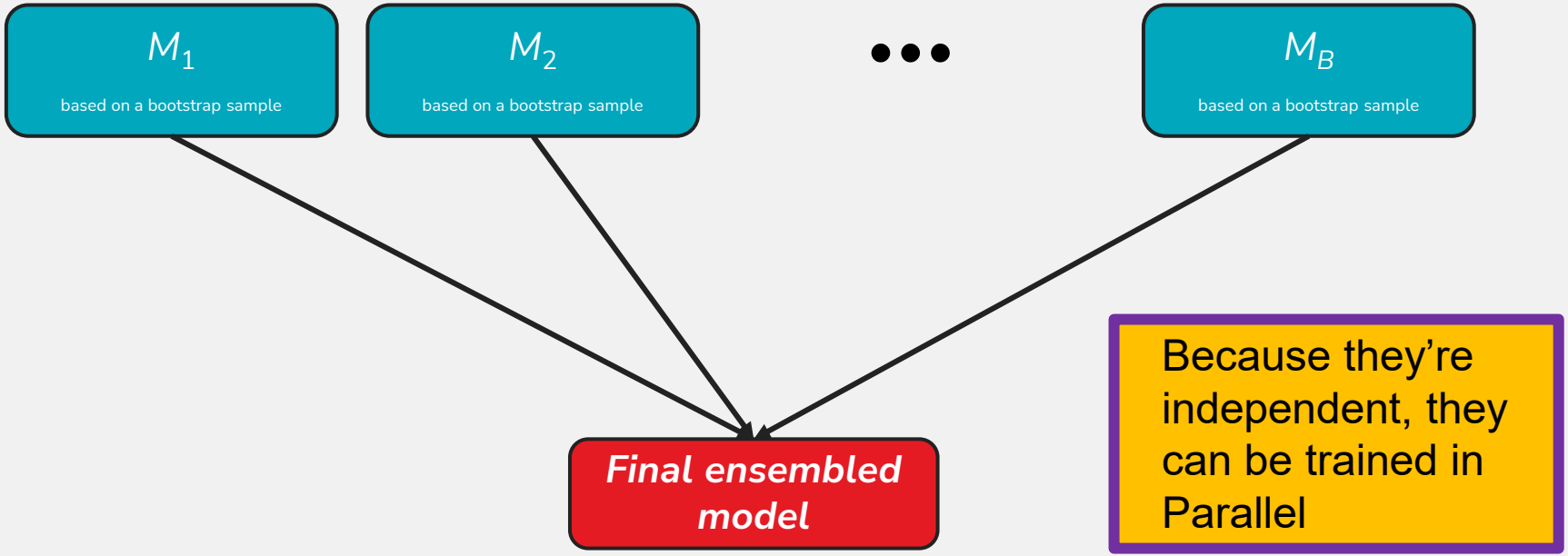
For **regression** tasks, the final prediction is the **mean** of these predictions.

For **classification** tasks, the final prediction is the **majority vote** of these predictions.

scikit-learn's classifiers actually average the predicted probabilities across trees ("**soft voting**") and pick the most likely class, which usually matches the majority vote

 © 2025 General Assembly

Bagging in a nutshell

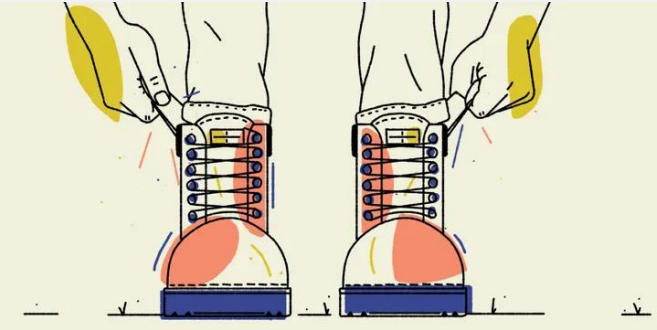


 © 2025 General Assembly

How do we take samples of data?

We use **Bootstrapping**.

A **bootstrap sample** is a **random sample with replacement**. It has the same size as the original sample but might duplicate some of the original observations.



a bootstrap sample contains about **63%** of the distinct rows, and about **37%** are left out

*Sidepoint: the **Bootstrap Technique** is a powerful resampling method used in statistics to estimate the sampling distribution of a statistic (like a mean, median, regression coefficient, etc.) by repeatedly sampling with replacement from the observed data.*

Source

 © 2025 General Assembly

For example...

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & x_{13} & x_{14} & x_{15} & x_{16} \\ x_{21} & x_{22} & x_{23} & x_{24} & x_{25} & x_{26} \\ x_{31} & x_{32} & x_{33} & x_{34} & x_{35} & x_{36} \\ x_{41} & x_{42} & x_{43} & x_{44} & x_{45} & x_{46} \\ x_{51} & x_{52} & x_{53} & x_{54} & x_{55} & x_{56} \\ x_{61} & x_{62} & x_{63} & x_{64} & x_{65} & x_{66} \\ x_{71} & x_{72} & x_{73} & x_{74} & x_{75} & x_{76} \\ x_{81} & x_{82} & x_{83} & x_{84} & x_{85} & x_{86} \\ x_{91} & x_{92} & x_{93} & x_{94} & x_{95} & x_{96} \end{bmatrix}$$

Our data set has:

- 9 observations
- 6 features

 © 2025 General Assembly

Bootstrap Simulation... Go!

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & x_{13} & x_{14} & x_{15} & x_{16} \\ x_{21} & x_{22} & x_{23} & x_{24} & x_{25} & x_{26} \\ x_{31} & x_{32} & x_{33} & x_{34} & x_{35} & x_{36} \\ x_{41} & x_{42} & x_{43} & x_{44} & x_{45} & x_{46} \\ x_{51} & x_{52} & x_{53} & x_{54} & x_{55} & x_{56} \\ x_{61} & x_{62} & x_{63} & x_{64} & x_{65} & x_{66} \\ x_{71} & x_{72} & x_{73} & x_{74} & x_{75} & x_{76} \\ x_{81} & x_{82} & x_{83} & x_{84} & x_{85} & x_{86} \\ x_{91} & x_{92} & x_{93} & x_{94} & x_{95} & x_{96} \end{bmatrix}$$

x2

x2

x2

x2

Our bootstrap sample has:

- 9 observations
- 6 features

 © 2025 General Assembly

Bagging

Pros:

- Ensembles trees to reduce their extremely high variance
- Averaging unbiased models results in an unbiased model... with lower variance!

Cons:

- Since the bootstrapped models are based on largely the same data, the resulting models will be *correlated!*
- Averaging correlated variables limits how much we reduce the variance by!

 © 2025 General Assembly

Evaluating models

Out-of-sample-error

Estimating Out-of-Sample Error

For bagged models, out-of-sample error can be estimated without using **train/test split** or **cross-validation**!

For each tree, the **unused observations** are called "out-of-bag" observations.

a bootstrap sample contains about **63%** of the distinct rows, and about **37%** are left out

 © 2025 General Assembly

Calculating "out-of-bag error"

1. For each observation in the training data, predict its response value using **only** the trees in which that observation was out-of-bag.
2. Average those predictions (for regression) or take a vote (for classification).
3. Compare all predictions to the actual response values in order to compute the out-of-bag error.

When **B** is sufficiently large, the (OOB) **out-of-bag error** is an accurate estimate of **out-of-sample error**.

 © 2025 General Assembly

Bagging + OOB error

Let's see it for ourselves! Parts 1–2

→ Open **02-ensemble-models-solution.ipynb**



 © 2025 General Assembly

Random Forests

A specific bagging method

The Random Forest Model

Random forest seeks to improve the variance reduction of bagging by decorrelating the tree without adding too much variance back.



 © 2025 General Assembly

The Random Forest Model - the extra randomness

Random forest seeks to improve the variance reduction of bagging by decorrelating the tree without adding too much variance back.

We do this by adding another source of randomness:

Before each split, select a random subset of the input variables as candidates for splitting.

(Citation: ESL: Elements of Statistical Learning)



 © 2025 General Assembly

How many variables to select at each split?

The original authors of the random forest (Breiman et al, 2001) recommend:

- $\sqrt{p}$ for classification with minimum node size of 1 (where p is the number of features)
- $p/3$ for regression with minimum node size of 5

And these recommendations are generally good. The authors of ESL, on the other hand, **recommend you treat them as tuning parameters** as the best value will depend on the problem.

 © 2025 General Assembly

What's the point?

Suppose there is one **very strong feature** in the data set. When using bagged trees, most of the trees will use that feature as the top split, resulting in an ensemble of similar trees that are **highly correlated**.

Averaging highly correlated quantities does not significantly reduce variance (which is the entire goal of bagging).

By randomly leaving out candidate features from each split, **random forests "decorrelate" the trees** to the extent that the averaging process can reduce the variance of the resulting model.

Example: a panel of experts who all read the same newspaper vs ones who read different sources

 © 2025 General Assembly

Random Forest

Where it's different to bagging

The steps for creating a Random Forest model are very similar to Bagging:

1. Get a bootstrap sample
2. Fit a decision tree:
    a. Each split in the tree is based on a **random** subset of m features from all p predictors
    b. Only one of the m features is used for the split
3. Repeating 1 and 2 many times (B = 100 or 500 times)
4. The final model is the average of all B models.

When $m = p$ the Random Forest is a vanilla Bagging model

 © 2025 General Assembly

Random Forest Simulation... Go!

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & x_{13} & x_{14} & x_{15} & x_{16} \\ x_{21} & x_{22} & x_{23} & x_{24} & x_{25} & x_{26} \\ x_{31} & x_{32} & x_{33} & x_{34} & x_{35} & x_{36} \\ x_{41} & x_{42} & x_{43} & x_{44} & x_{45} & x_{46} \\ x_{51} & x_{52} & x_{53} & x_{54} & x_{55} & x_{56} \\ x_{61} & x_{62} & x_{63} & x_{64} & x_{65} & x_{66} \\ x_{71} & x_{72} & x_{73} & x_{74} & x_{75} & x_{76} \\ x_{81} & x_{82} & x_{83} & x_{84} & x_{85} & x_{86} \\ x_{91} & x_{92} & x_{93} & x_{94} & x_{95} & x_{96} \end{bmatrix}$$

x2

x2

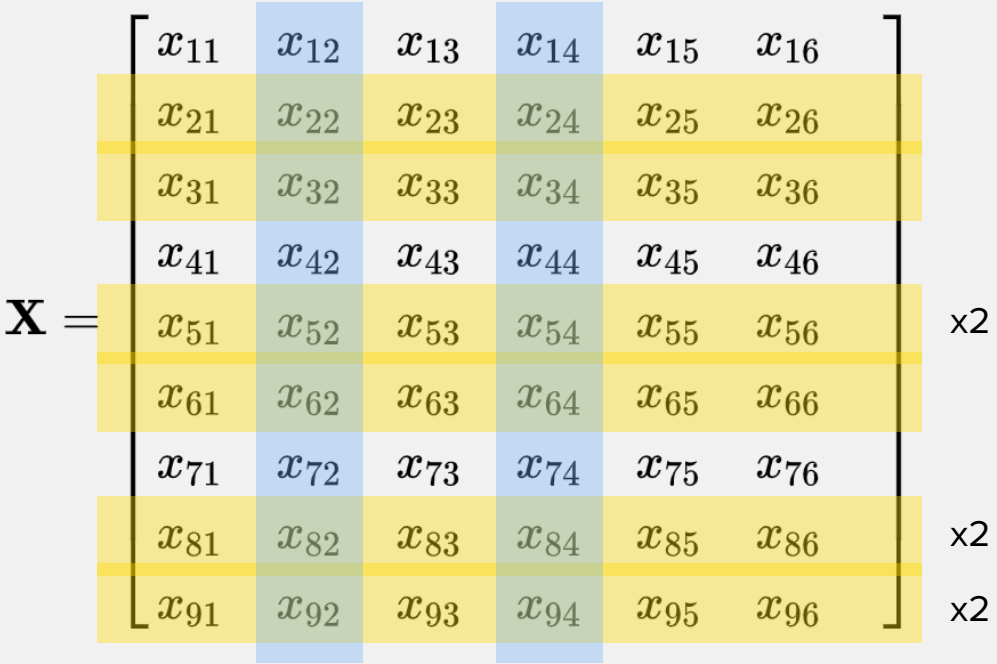
x2

Our bootstrap sample has:

- 9 observations
- 6 features

 © 2025 General Assembly

Random Forest Simulation... Go!



Our bootstrap sample has:

- 9 observations
- 6 features

 © 2025 General Assembly

Estimating Feature Importance

Bagging increases **predictive accuracy** but decreases **model interpretability** because it's no longer possible to visualize the tree to understand the importance of each feature. This is sometimes called a **black box model**.

But we can still obtain an overall summary of **feature importance** from bagged models:

1. **Bagged regression trees**: Calculate the total amount that **MSE** decreases due to splits over a given feature, averaged over all trees
2. **Bagged classification trees:** Calculate the total amount that **Gini index** decreases due to splits over a given feature, averaged over all trees

*biased towards continuous / many-valued features; computed on training data. `permutation_importance` on test data is the more reliable alternative.

 © 2025 General Assembly

Random Forests

Pros:

- Their performance is competitive with the best supervised learning methods.
- They provide a more reliable estimate of feature importance.
- They allow you to estimate out-of-sample error without using train/test split or cross-validation. *(But you still need a TTS split because you're tuning parameters!)*

Cons:

- They are less interpretable.
- They are slower to train.
- They are slower to predict.

 © 2025 General Assembly

Random Forests

Let's see it for ourselves! Part 3

→ Open **02-ensemble-models-solution.ipynb**



 © 2025 General Assembly

Boosting

Focus on fixing our mistakes!

How would you tutor your friend?

Imagine you're tutoring a friend who keeps getting math problems wrong. You notice a pattern: they mess up fractions, but not multiplication.

How would you help them improve?



Bagging and Random Forests

An added bonus is that these ensemble models are **parallelizable**: the individual base models (learners) can be trained independently and simultaneously across multiple processors or machines.

Why is that a benefit?



 © 2025 General Assembly

Boosting

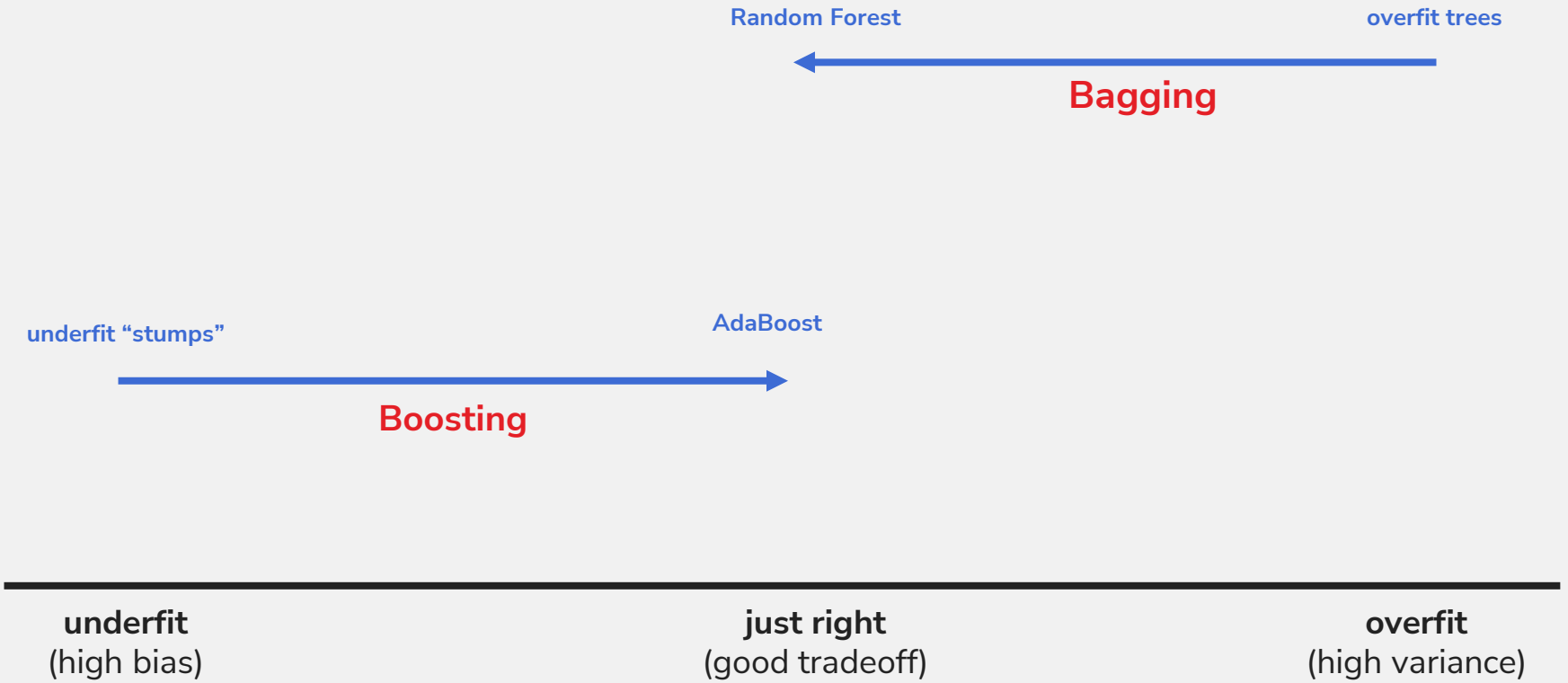
In contrast to bagging, **boosting** is another ensemble technique that solves the opposite problem:

We're going to take **weak learners** *(a model only slightly better than guessing)* and successively **boost** them to **reduce bias.**

Typically, these weak learners are going to be very high-bias decision trees. Specifically, they're most often 1-split **decision stumps***. (i.e., `max_depth=1`)

** Typical for AdaBoost*

 © 2025 General Assembly



 © 2025 General Assembly

AdaBoost

Math inbound

AdaBoost

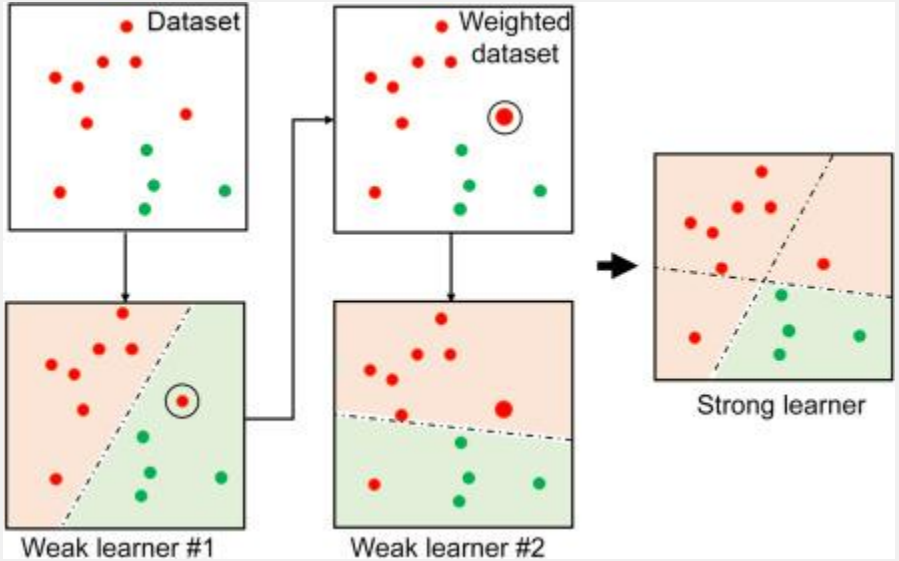
The “simplest” (lol) form of boosting is known as the **AdaBoost**.

The general idea of boosting is to fit successive models that each improve upon the one before them.

After each model step in the AdaBoost, the **next model pays more attention to the parts of the data it got the most incorrect**. By focusing special attention on these parts, it spends most of its time learning the hardest parts of your data rather than the easiest, most separable parts.

 © 2025 General Assembly

Visually, this is what AdaBoost looks like



Source

 © 2025 General Assembly

AdaBoost Algorithm (as math-free as possible)

1. Initialize the observation weights **w** to each be 1/n (i.e., all weighted the same)
2. For $m = 1, \ldots, M$:
    a. Fit a ***weighted*** underfit model G_m
    b. Compute the ***weighted*** misclassification rate
    c. Compute the model weight α_m that determines this model's overall importance to the ensemble
    d. Reweight the observations as weight vector **w**. This is a vector in which the "easy" observations have a low weight and the "hard" observations get a large weight
3. The final model G is the weighted amalgamation of all the G_ms and the α_ms.

 © 2025 General Assembly

AdaBoost

Let's see it for ourselves! Part 4

→ Open **02-ensemble-models-solution.ipynb**



 © 2025 General Assembly

Gradient Boost

(oh God there's more math and it's worse somehow)

AdaBoost vs Gradient Boost

Both are similar boosting techniques, but gradient boost takes a slightly different approach.

Where **AdaBoost** improves successive models by focusing more on the hard-to-reach parts of our data...

Gradient Boost focuses on the observations with the worst residuals and successively corrects the model to "fix" them.

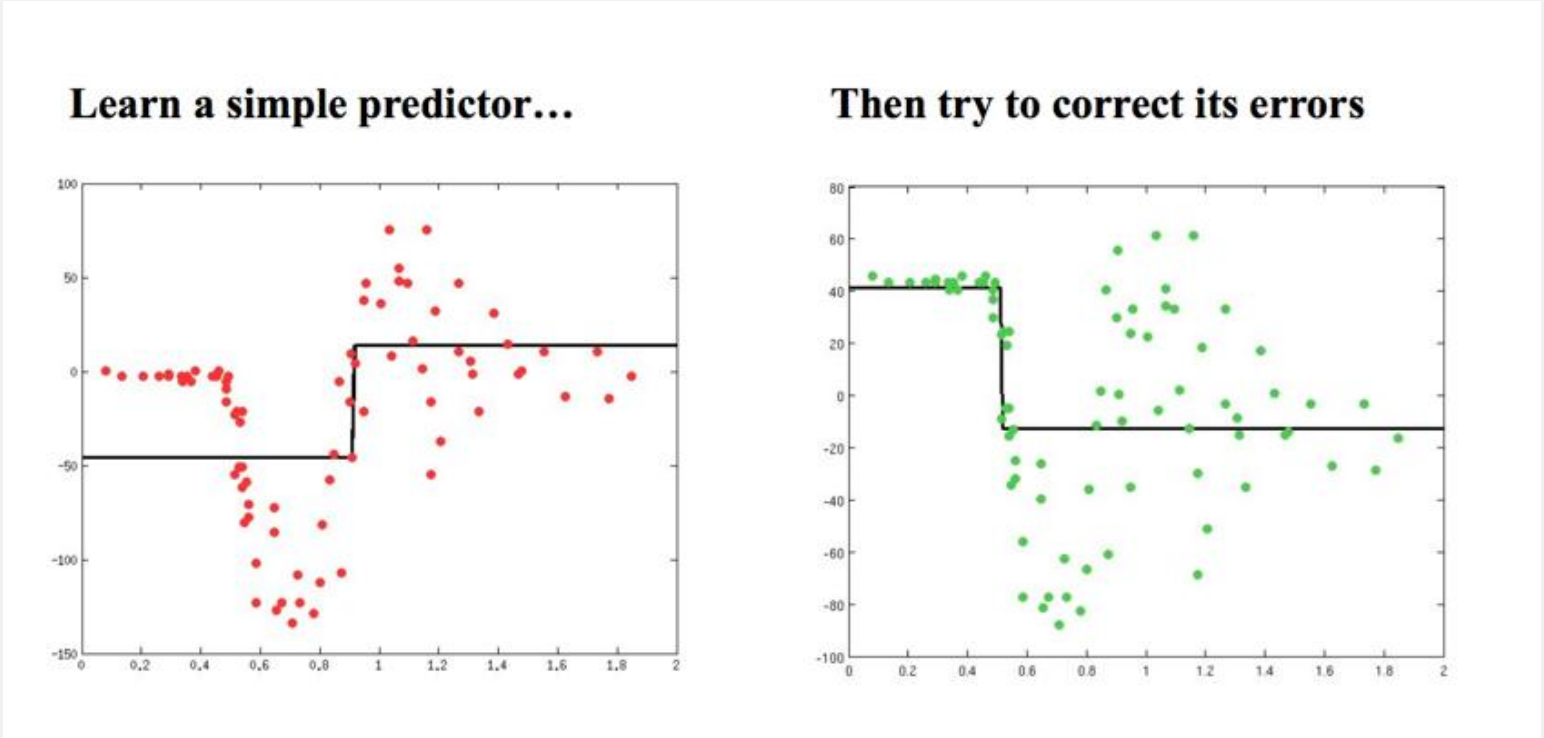
 © 2025 General Assembly

The Gradient Tree Boost Algorithm (math lovers keep OUT)

1. Initialize the base model f_0 to be the null model.
2. For $m = 1, \ldots, M$:
    a. Compute the ~~gradient of the loss function~~ "residuals", **r**.
        i. There's a special way of computing residuals for classification problems called **deviance**, and let us never speak of it again!
    b. Fit an underfit tree model, except with **r** instead of **y** as the response variable.
    c. For each terminal region (i.e., partition of feature space) computed in 2b, compute a new residual γ that gets you the best loss function with respect to $f_{m-1} + \gamma$ (i.e., this is readjusting the predictions based on how bad they were)
    d. Update $f_m = f_{m-1}$ + adjustments involving γ
3. The final model is just f_M

 © 2025 General Assembly

A quick little visual aid of GradBoost



 © 2025 General Assembly

A note on these algorithms...

We're getting to the point where the algorithms themselves are far beyond the mathematical scope of our bootcamp.

For us, the key things to remember are the gists of why these algorithms do what they do, and the basics of how.

Failing that, we can just ham-fist them in Python, so...

 © 2025 General Assembly

XGBoost

Another one???

eXtreme Gradient Boosting

Maybe the most popular form of boosting today is the **Extreme Gradient Boosting** algorithm, aka **XGBoost**.

It is algorithmically similar to **gradient boost** but has been shown to generally get better results.

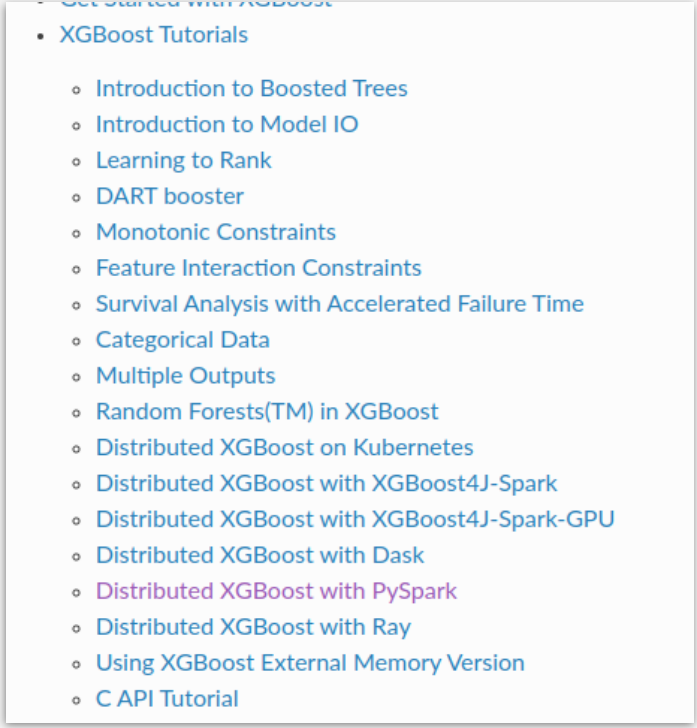
Alternatives with similar performance: **LightGBM**, **CatBoost**, and scikit-learn's **HistGradientBoosting**.

 © 2025 General Assembly

XGBoost

More importantly, however, is that the algorithm is engineered to be **parallelizable**, and its implementation in Python can also be **distributed**!

This makes it a very popular model for Big Data problems.



 © 2025 General Assembly

XGBoost: The Pros & Cons

Pros:

- Really good, Kaggle-winning predictions
- Optimized, regularized gradient boosting
- Handles missing data natively!
- Scales to massive data

Cons:

- More prone to overfitting, even among tree-based models
- Can be *very* slow to fit on large data

 © 2025 General Assembly

Implementing XGBoost

XGBoost is not implemented in **`scikit-learn`**. Instead, it has its own library. So in order to use it, we'll need to:

```
pip install xgboost
```

 © 2025 General Assembly

Gradient Boosting & XGBoost

Let's see it for ourselves! Parts 5–6

→ Open **02-ensemble-models-solution.ipynb**



 © 2025 General Assembly

Bring It Home

Bagging vs Random Forests vs Boosting

Ensemble Method	Learner Focus	Goal	Typical Learner
Bagging	Independent	Reduce Variance	Deep Trees
Random Forest	Independent + random features per split	Reduce variance further	Deep Trees
Boosting	Sequential	Reduce Bias	Stumps or small trees

 © 2025 General Assembly

Boosting

Pros: top-tier accuracy and all the convenience of trees.
Cons: hard to interpret, and slower, because the trees must be built one after another.

Pros:

- Top-tier performance
- You get all the same benefits associated with tree-based models

Cons:

- Interpretation is reduced. *(feature importance, partial dependence plots and SHAP all work on boosted trees)*
- Computational complexity is the highest we've ever seen - can cause models to fit slowly!
- Because of serial nature of algorithm, parallelization is impossible *(**across** rounds is impossible, but **within** each round it's normal)*

 © 2025 General Assembly

Which ensemble should I use?

Start with a random forest: strong, stable, little tuning.

Need the last bit of accuracy? Try gradient boosting; tune the learning rate and number of trees.

Small or noisy data? Random forests are more forgiving.

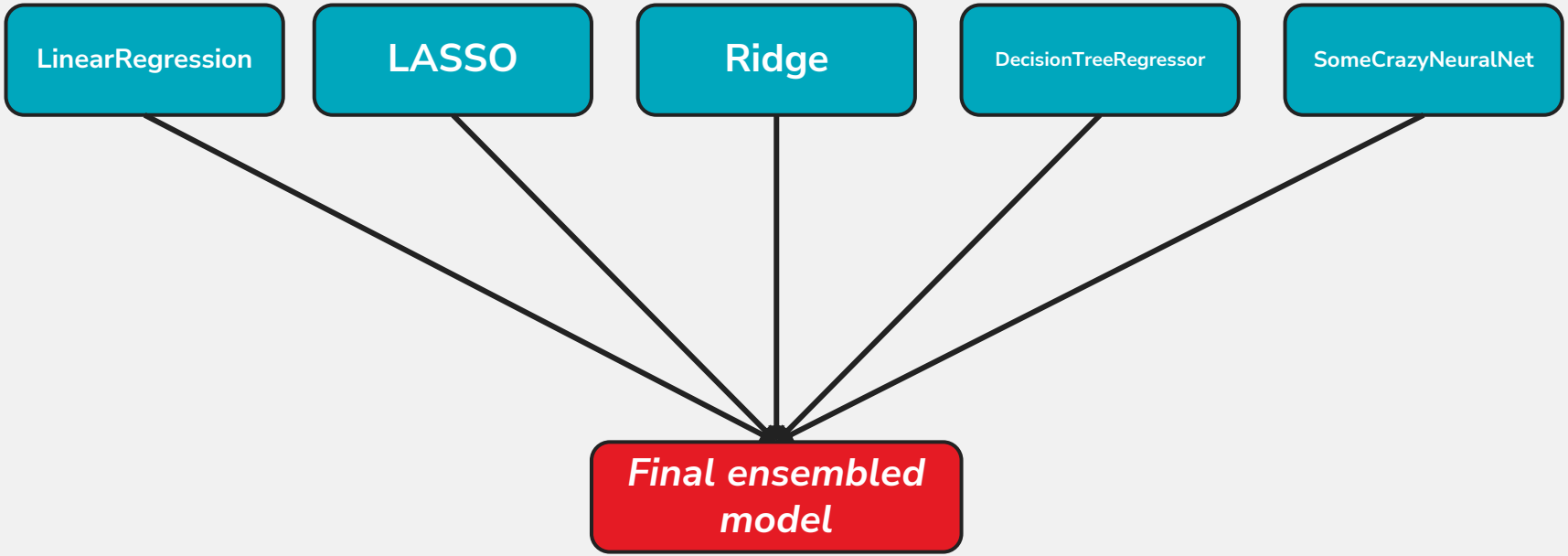
Must explain every decision? Use a single tree or logistic regression.

Always: compare with a simple baseline.



 © 2025 General Assembly

Stacking involves arbitrary models ensembled together



 © 2025 General Assembly

Key takeaways



Many trees beat one

Bagging averages away variance; boosting reduces bias



Forests: a safe default

Random splits decorrelate the trees; OOB error is a free test score



Boosting fixes mistakes

Often the most accurate; tune the learning rate and number of trees

 © 2025 General Assembly

Ok, but why trees?

Graphic taken from our Rosarium philosophorum, ***The Elements of Statistical Learning***, by Hastie, Tibshirani, and Friedman (praise be unto them)

TABLE 10.1. *Some characteristics of different learning methods. Key:* ▲ *= good,* ◆ *=fair, and* ▼ *=poor.*

Characteristic	Neural Nets	SVM	Trees	MARS	k-NN, Kernels
Natural handling of data of "mixed" type	▼	▼	▲	▲	▼
Handling of missing values	▼	▼	▲	▲	▲
Robustness to outliers in input space	▼	▼	▲	▼	▲
Insensitive to monotone transformations of inputs	▼	▼	▲	▼	▼
Computational scalability (large N)	▼	▼	▲	▲	▼
Ability to deal with irrelevant inputs	▼	▼	▲	▲	▼
Ability to extract linear combinations of features	▲	▲	▼	▼	◆
Interpretability	▼	▼	◆	▲	▼
Predictive power	▲	▲	▼	◆	▲

 © 2025 General Assembly

AdaBoost (mathfull notation)

When talking about boosting (and also for SVM models), when we talk about binary classification, we let $y \in \{-1, 1\}$ instead of the usual $y \in \{0, 1\}$ so we can take advantage of a few math tricks.

In what follows...

- $G_m(x)$ is a model, and so it can take only values -1 and 1
- $I(.)$ is the indicator function. ie, $I(A)$ is 1 if A is true and 0 otherwise.
- $\text{sign}(.)$ is the "signum" function. ie, $\text{sign}(x) = 1$ if x is positive and -1 if it's negative.

 © 2025 General Assembly

AdaBoost Algorithm (for all you math nerds out there)

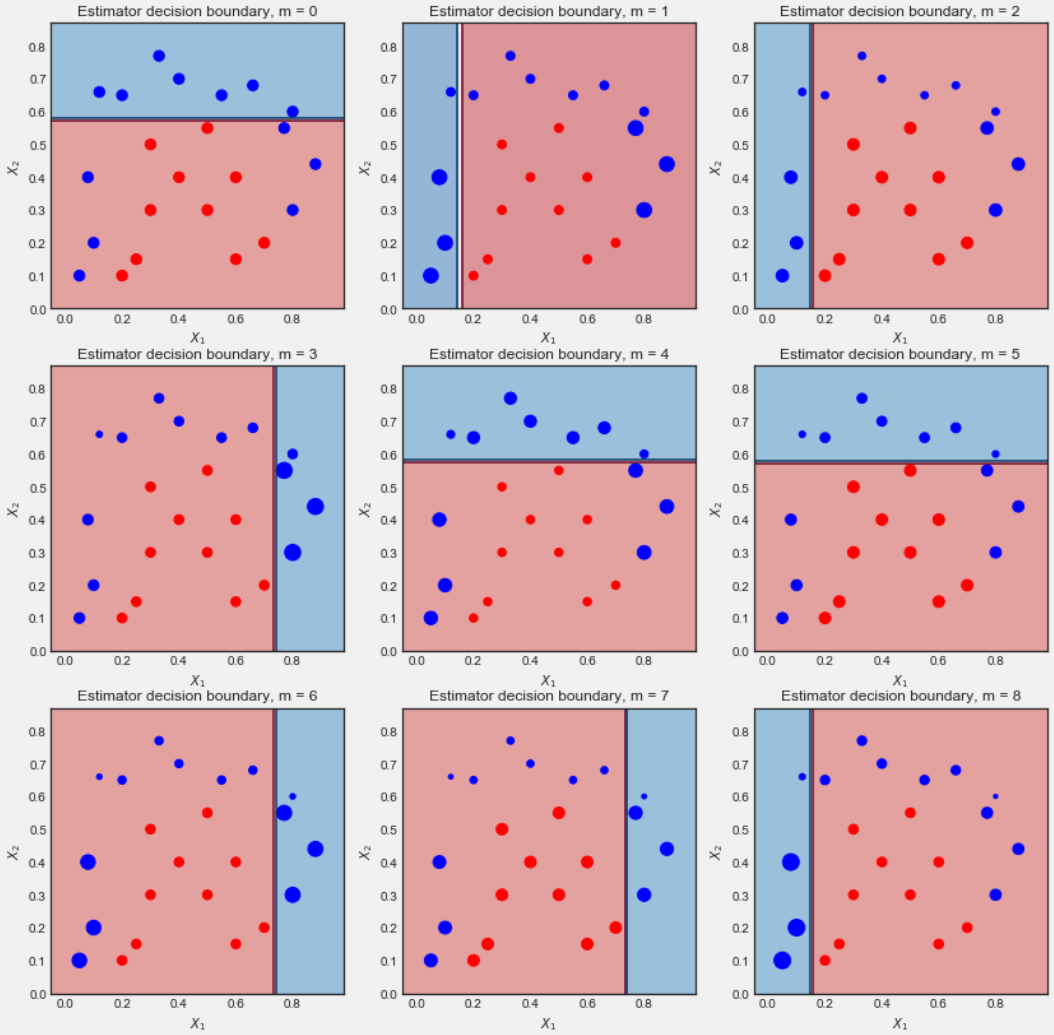
Algorithm 10.1 *AdaBoost.M1.*

1. Initialize the observation weights $w_i = 1/N,\ i = 1, 2, \ldots, N$.
2. For $m = 1$ to M:
   (a) Fit a classifier $G_m(x)$ to the training data using weights w_i.
   (b) Compute
   $$\text{err}_m = \frac{\sum_{i=1}^{N} w_i I(y_i \neq G_m(x_i))}{\sum_{i=1}^{N} w_i}.$$
   (c) Compute $\alpha_m = \log((1 - \text{err}_m)/\text{err}_m)$.
   (d) Set $w_i \leftarrow w_i \cdot \exp[\alpha_m \cdot I(y_i \neq G_m(x_i))],\ i = 1, 2, \ldots, N$.
3. Output $G(x) = \text{sign}\left[\sum_{m=1}^{M} \alpha_m G_m(x)\right]$.

(As always, clipped directly from ESL)

 © 2025 General Assembly

AdaBoost Viz



 © 2025 General Assembly

Gradient Tree Boosting algorithm, behold!

Algorithm 10.3 *Gradient Tree Boosting Algorithm.*

1. Initialize $f_0(x) = \arg\min_\gamma \sum_{i=1}^{N} L(y_i, \gamma)$.
2. For $m = 1$ to M:
   (a) For $i = 1, 2, \ldots, N$ compute

$$r_{im} = -\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f=f_{m-1}}.$$

   (b) Fit a regression tree to the targets r_{im} giving terminal regions R_{jm}, $j = 1, 2, \ldots, J_m$.
   (c) For $j = 1, 2, \ldots, J_m$ compute

$$\gamma_{jm} = \arg\min_\gamma \sum_{x_i \in R_{jm}} L\left(y_i, f_{m-1}(x_i) + \gamma\right).$$

   (d) Update $f_m(x) = f_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$.
3. Output $\hat{f}(x) = f_M(x)$.

Lifted from ESL

 © 2025 General Assembly


GA
GENERAL ASSEMBLY